Autonomous Navigation of Drone Through Real Environment Using Reinforcement Learning and Transfer Learning

Ruben McWilliams
MS Data Science Student
rsm5286@g.rit.edu

Tolu Olatunbosun
MS Data Science Student
tao5634@g.rit.edu

Shrinivas Sampath Muthupalaniyappan
MS Data Science Student
sm3533@g.rit.edu

Blake Donley
BS/MS Computer Engineering Student
bad6983@g.rit.edu

Advisor: Dr. Travis Desell (Email: tjdvse@rit.edu)
Sponsor: National Science Foundation (Email: info@nsf.gov)

1 Abstract

In many robotics task, such as autonomous obstacle avoidance using drone, the goal is to fly in any given environment without hitting any obstacles and fly for as long as possible. Previous approaches have focused more only on making the drone fly for a longer time and not solved any socially impactful problems. Hence, it is important to make research impactful by offering social community. A key challenge for the task is to use reinforcement learning to train the drone in a simulated environment and apply transfer learning to a physical drone. To make it novel, we are trying to solve the monitoring and surveillance problem by making our drone not only avoid obstacles using reinforcement and transfer learning but also to monitor objects once the drone detects the objects. This results in an open source repository that anyone can use as a starting point for drone monitoring and surveillance tasks. So far, we had implemented a preliminary reinforcement learning code to make our drone fly in certain altitude range where the reward will be maximized.

2 Introduction

Autonomous drones solve problems through powered navigation and operational software to survey and inspect regions and potential problems with efficiency. This project presents a challenge worthy of being an applied data science project as it requires the use of simulation software in order to simulate the process of a drone monitoring objects in an environment, the project requires knowledge of flying drones and the laws that go along with flying them, it requires knowledge of writing software that will fly the drone autonomously, and the project requires the training of a computer vision model in order to detect these specific objects. The minimum goal we want to achieve in order to be able to call our project a success is to program a drone to navigate its environment without collisions and to perform monitoring and surveillance of boxes in an environment. The utility of this project extends to issues with power lines, as the electric power sector tends to be very reactive rather than preventative which often means that they tend to wait until something goes wrong to fix power lines rather than incorporate preventative methods to prevent these problems from occurring in the first place. Consider the scenario where an organization monitors power lines. This can cost a lot of money when power lines get damaged, need repair, and many people lose power. It can also be extremely dangerous such as in the case of a short circuit, where a fire could start, or the case of a fallen power line, which could easily kill someone. This could also cause legal issues, which could cost a lot of money. Through our research, the field will have a way to train drones to make decisions without a pilot, and to detect objects of interest using computer vision. There has been research in this area before, though they are mainly in the act of seeing what is possible for autonomous drone software, and often not in particularly applied problems.

This programming project could be extended to apply unmanned aerial vehicles monitoring

and surveillance to different sectors. In particular, computer vision for monitoring items in an environment allows for inspection of a large component to urban life and modern industry. This can include inspections using drones provides a safe alternative to checking power lines as people do not have to climb them to check them anymore, preventing them from having to risk their lives. It also allows for close up inspection, unlike using helicopters. Broader impacts of this project enables the application of surveillance in various industries. In the papers we read, these industries included: defense, wildlife ecology, and in public and private sectors, among others. The concept of regular surveillance can be applied to other industries as well, catching problems as early as possible. This gives agencies, businesses, and citizens the ability to rectify damage to infrastructure long before it becomes catastrophic. Catching issues early makes it much cheaper to fix rather than catching issues later on, which is another benefit of this approach. Detecting issues early on also saves lives as it prevents major problems from occurring such as a fallen power line wire. This also reduces litigation regarding insurance policy, and more. Autonomous Drones, Reinforcement Learning, and Computer Vision expands beyond the realm of monitoring items in a simulated or enclosed space into a broader notion of enhanced life quality and efficacy. The rest of this paper details the machine learning techniques we implement during this project, and how we plan to utilize the drone and train it to complete our objectives.

3 Related Work

3.1 Reinforcement Learning and Transfer Learning:

Hierarchical Memory System With STT-MRAM and SRAM to Support Transfer and Real-Time Reinforcement Learning in Autonomous Drones

There is currently a system called E2E RL which allows drones to fly autonomously in their environment using reinforcement learning. The idea is to train a drone in the real world using reinforcement learning by letting it fly around, meanwhile training all of the layers of a convolutional neural network. [1] The problem with this approach is that it has high latency and high energy dissipation. [2] The goal of this paper written by Insik Yoon, Malik Aqeel Anwar, Rajiv V. Joshi, Titash Rakshit, and Arijit Raychowdhury, was to create a system with comparable performance to E2E RL which did not have such high latency and energy requirements. [2] We utilize this paper in order to get a better understanding of how Aqeel Anwar's repository works since we utilize this repository in our project. However, our work adds to this work as we are doing monitoring and surveillance (specifically, of boxes) which this paper does not, the paper only cares about how long the drone can fly in its environment without crashing.

Distributed Reinforcement Learning for Flexible UAV Swarm Control with Transfer Learning Capabilities

Drones are being used more for remote area monitoring and surveillance due to price reductions and improvements in the capabilities of drones [3]. Drones in a swarm need to successfully navigate their environment, working cooperatively. They need to identify and monitor interesting targets and try to keep their movements to a minimum [3]. The paper proposes a technique that uses distributed Reinforcement Learning which performs better than a computationally intensive look-ahead heuristic [3]. The paper also demonstrates the ability of agents to adapt to a new scenario after being trained for a specific scenario, with a minimal amount of additional training [3]. We utilize this paper in order to get an idea of what has been done in terms of monitoring and surveillance using drones since we are doing this in our project. The paper also talks about using Transfer learning to do this, and we utilize transfer learning in our project. Our paper is different from this paper since we do not use drone swarming in our project, we only use a single drone. We are also simulating our drone in a realistic Unreal Engine environment and then testing in a real environment, this paper does neither of those things.

Autonomous Navigation via Deep Reinforcement Learning for Resource Constraint Edge Nodes Using Transfer Learning

The world is forever changing by the introduction of smart technologies and agile drones [4]. In this paper authors explains faster way of training the drone using Deep Reinforcement Learning (DRL) in simulated environment called Unreal Engine [5] and then the most important weights in the beginning of the layers are transferred using a method called transfer learning to the test environments and fine tuned for the last fully connected layers of the network [4]. The light weight drones are called resource constraint edge nodes according to the paper. Moreover they come up with additional computer requirements, increased power consumption and increased latency [4]. The contribution of the paper is an energy efficient approach for Reinforcement Learning based on drone autonomous navigation and a python based framework for drone related applications [4]. The authors approach was to use transfer learning which was primarily trained on a simulated environment called Unreal Engine [5] and once the drone was trained in the simulated environment the meta weights in convolution (CONV) layers are initialized by transferring in online training phase [4]. We read this paper because it discusses the use of Aqeel Anwar's repository which we utilize in our project. It also discusses the same workflow that we follow in our project where we train the drone in a simulated environment and then take those weights/biases and move them to a drone in the real world and then continue training. Our project is different however as we are utilizing multiple repositories, not just the one discussed in the paper. Also in this paper the drone is just trying to navigate its environment without crashing for as long as possible, we add to this by having our drone not only do this, but also monitor boxes by flying near them. This will result in a repository that anyone can use as a template for monitoring and surveillance tasks.

3.2 Reinforcement Learning

Deep Reinforcement Learning: A Brief Survey

Although Neural Networks are often utilized as the domain model, the specific mechanism that makes these actions possible is the advancement of Deep Reinforcement Learning (DRL) architectures. [6] Much like human learning experience, Reinforcement Learning mimics learning from live experiences in order to gain a sense of positive and negative outcomes from real interactions. Specifically, the agent in the learning environment is the principal decision maker. [7] All forces that interact with the agent constitute the environment, including stationary and mobile items the agent may engage. Reinforcement learning has existed from 1960 and it has been widely used in all the fields like robotics, chemistry and also making financial decisions. The key concept of RL is instead of hard coding the movement of robots like bend the knee and move forward. The RL finds the best action by trial and error method [8]. Few examples of Deep Reinforcement Learning were playing a range of Atari 2006 video games [9], alpha go by deepmind [10] and the achievement of IBM's Deep Blue in chess two decades earlier [11]. We read this paper to get a general idea of what reinforcement learning is and what has been done using it since we are utilizing reinforcement learning in our project.

Drone Navigation and Avoidance of Obstacles Through Deep Reinforcement Learning

In the paper "Drone Navigation and Avoidance of Obstacles Through Deep Reinforcement Learning" by Ender Cetin, Cristina Barrado, Guillem Munoz, and Enric Pastor, there was an exploration of utilizing drones for autonomous navigation which was enabled by Reinforcement learning. [7] The paper discusses using geo-fences as this virtual barrier around a geographical area which allows for creating constraints for a drone to establish grounds to produce rewards and negative feedback as the drone was trained using obstacles and taking actions in a defined action space. [7] We read this paper because it discusses training a drone using reinforcement learning to autonomously navigate an environment, which is what we are doing in our project

as well. The main difference between our project and this paper is that we are not utilizing a geo-fence as described in the paper.

Autonomous Drone Racing with Deep Reinforcement Learning

In the paper "Autonomous Drone Racing with Deep Reinforcement Learning" by Yunlong Song, Mats Steinweg, Elia Kaufmann, and Davide Scaramuzza, researchers underwent the task of racing drones in various virtual environments to update neural network policies using deep reinforcement learning. [16] By implementing parameters that account for quadrotor dynamics, the methodology takes into account different velocity vectors to manipulate the orientation of the quadrotor, the rotation of the body, the dimensional torque operating on the device, and the gravity vector and torque operating on it as it generates rotor thrusts to make different aerial maneuvers. [16] We read this paper because the researchers fly and train drones using reinforcement learning in a virtual environment, which is what we are doing in our project. This paper is different from our project, however, as we do not race our drone, we just allow it to fly around in its environment and learn from it using reinforcement learning.

Vision Based Drone Obstacle Avoidance by Deep Reinforcement Learning

In the paper "Vision Based Drone Obstacle Avoidance by Deep Reinforcement Learning" by Zhihan Xue and Tad Gonsalves, experiments were performed to continue training drones to perform obstacle avoidance by establishing the limits of supervised learning techniques, and by addressing those limits using reinforcement learning. Notably, this article navigates the "soft-actor-critic algorithm to help the drone operate using image data. [17] Namely, the experiments train the soft-actor-critic network using a neural network that updates with each epoch until the rewards from the launch result in a crash or reach the final goal. [17] We read this paper because it has to do with training a drone to perform obstacle avoidance using reinforcement learning, which is part of what we are doing since our drone will have to navigate its environment successfully by avoiding obstacles as well as monitoring the boxes by flying near them. Our project is different from this one however in that we are not using the soft-actor-critic algorithm.

3.3 Transfer Learning

A Survey on Deep Transfer Learning

Transfer learning is a key concept in deep learning, whenever we need to train a deep learning model for a different task transfer learning can be applied to reduce the time for training, let's say if you wanted to train a model to identify car you can use the features or also called weights from the first few layers in the truck model to your car model to save lot of your training time because first few layers always consist of edges, sharpening and colors, which are really high level features and can be reused in the car model as well instead of retraining again [18]. The last few fully connected layers can be trained again for our car model to save a lot of training time [18]. The previous example is one type of transfer learning. Another significant reason to use transfer learning is it doesn't depend on the test data [18]. There are currently four different types of transfer learning i)Instances based transfer learning - If we want to use specific instances from the source domain for the transfer to the target domain, then we can use the instance based transfer learning to utilize specific instances with appropriate weights [18]. ii) Mapping based deep transfer learning - If we have completely two different domains but these domains can be combined together to produce a new data space which might be relevant together [18]. iii)Network based transfer learning - These types of transfer learning are more common nowadays, the relevant features are used from the front layers in convolutional to the newly developed network and the last few fully connected layers are trained again with the newly added weights to fine tune our model [18]. iv) Adversarial based deep transfer learning - The source domain and target domain are fed into an adversarial network which takes both the source and target domain high level features to discriminate it. If the adversarial network performs badly that means the source domain can be transferred to the target domain or vice versa [18]. The drawback of the paper

was it lacked examples for the different types of transfer learning.

3.4 Computer Vision

Intelligent Monitoring and Inspection of Power Line Components Powered by UAVs and Deep Learning

Drones have a huge potential in being utilized for power line monitoring and inspection [19]. This paper develops an effective model that is able to outperform other alternative models [19]. They test the drone and find that it is capable of detecting power line faults quickly and accurately [19]. The paper's approach was to perform data augmentation on the data they already had in order to balance out the classes and also to generate more data [19]. The authors built a multi stage pipeline where an object detection model is first used to detect a power mast, and then the model zooms into that area [19]. Next, it uses an object detection model to detect all of the components, it then loops through each one and zooms into it then uses a classification model to detect whether a fault is present or not [19]. The classification models in the pipeline are built by fine-tuning the ResNet50_cvgj model and the object detection models are Single Shot multibox detectors (SSDs) [19]. As mentioned, this very interesting pipeline is able to outperform other alternative models [19]. We read this paper because we needed to decide what kind of object detection model we were going to use for detecting boxes. This paper provided a thorough analysis of different object detection models and determined that the Single Shot Detector was the best, thus this is the one we went with for our project. Other than this, however, our project is very different from the paper as the paper focuses on power line monitoring and inspection (they fly the drone manually with a controller then do offline analysis) and our project focuses on the autonomous monitoring and surveillance of boxes in an environment.

4 Approach/Methodology

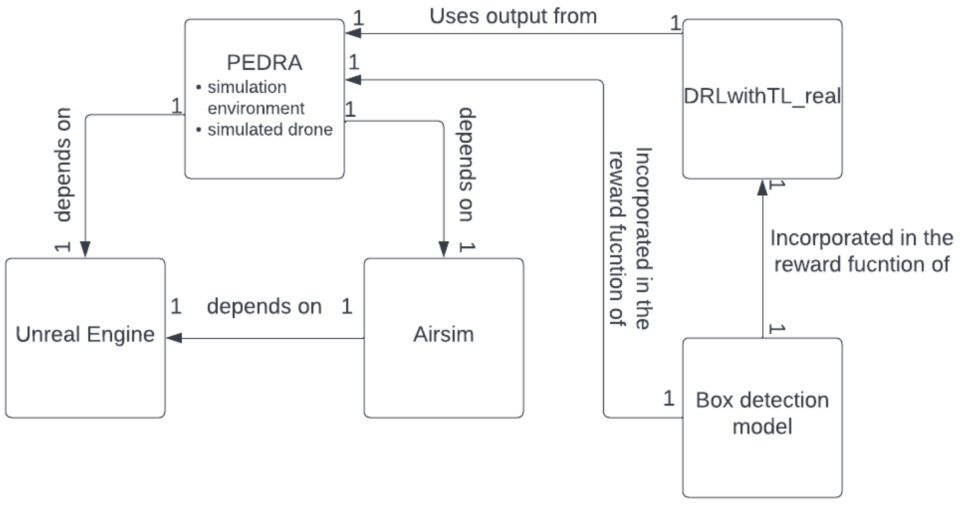


Figure 1: **Domain Model Diagram**

Explanation/description of above domain model diagram: PEDRA is a GitHub repository that we use to train a drone to fly in a simulated environment using reinforcement learning. PEDRA allows users to choose a drone, an environment, and a reinforcement learning algorithm for training. We chose the DJI Mavic drone because PEDRA doesn't have a Tello drone option, the outdoor forest environment (we wanted an environment that we could also find nearby our

campus so we could do transfer learning later), and we chose the DeepREINFORCE algorithm because it is the default recommended algorithm by PEDRA and the output of this algorithm is more interpretable than that of the other algorithm in PEDRA. Once reinforcement learning has finished its training process we have an output reinforcement learning neural network model with trained weights/biases. Next, we use a different repository called DRLwithTL_real to train a DJI Tello drone to navigate a real world environment using reinforcement learning. We take the weights/biases from the reinforcement learning neural network outputted by PEDRA and use these weights/biases to initialize the weights/biases of our network in DRLwithTL_real then training this reinforcement neural network further in the real world in a outdoor forest environment (it needs to match the PEDRA environment), thereby performing transfer learning. We perform transfer learning in order to speed up convergence. PEDRA depends on AirSim (an open-source, cross platform simulator for drone for vehicles such as cars and various other objects, built on Epic Games' proprietary Unreal Engine 4 as a platform for AI research [20]) in order to run, which in turn depends on Unreal Engine (a 3D computer graphics game engine developed by Epic Games [21]). PEDRA also depends on Unreal Engine. Once we see that everything previously stated works correctly, we add a new component: the box detection model. We will train this model to detect boxes in the real world. Then, we will incorporate this box detection model in the reward functions of PEDRA and DRLwithTL_real so that if the drone goes towards a box, it gets a positive reward, and thus will be trained to monitor the boxes.

Many pre-existing approaches exist for using transfer learning and reinforcement learning. Our approaches do not contain any new reinforcement learning algorithms or innovative transfer learning methods, with the possible exception of our addition of using the box detection model inside of our reward function; rather, we utilize the available code from the following repositories (DRLwithTL_real and PEDRA). But the cost of using these old repositories that haven't been updated in a while comes with great difficulty. First, most of the library versions in those repositories are outdated and all dependencies need to be resolved by upgrading to specific versions. Second, few instances of the implemented library methods need to be refactored manually depending upon newer libraries' requirements. Third, the neural networks available in two repositories should be adaptable for transfer learning. This may cost us to rewrite the entire neural network on both the repositories to make it even more challenging. Fourth, to make our problem more challenging and novel, we are introducing boxes. Apart from navigating the drone using Reinforcement Learning and aiding training using Transfer Learning, we will rewrite the code for the reward function on DRLwithTL_real and PEDRA repositories to make the drone navigate through the obstacles and also to search for the box. Our proposed method differs from previous work in the following aspects:

a) Redevelop existing framework for reinforcement learning using Airsim and Unreal Engine (i.e redevelopment points to installing latest versions of libraries, solving the dependencies between each library, refactoring the code as needed). b) A novel independent neural network model is trained for box detection and used in the reward function of the two repositories to provide reward when the drone approaches the boxes.

Out-of-the-box PEDRA allows you to train a drone to fly around an environment without crashing for as long as possible. We improve upon this so that instead of just flying around aimlessly, the drone completes the task of monitoring the boxes.

We will use a reinforcement learning network because we need to train the drone so it can navigate without human input using reward and penalty parameters. Upon taking successful steps towards the intended outcome we seek, the agent will be rewarded for its behavior and therefore continue it, and vice versa in the event it takes actions that negatively affect the intended outcome (the penalty). More specifically, if the drone is going towards a wall, it will get a negative reward, thereby learning how to avoid hitting walls/obstacles. Also, if the drone is flying towards a box, it will get a positive reward, thereby learning to fly near boxes (monitoring them). We will initialize

our box detection model with ImageNet weights/biases in order to speed up its convergence.

Our decisions relate to our domain model because they depict the way the softwares and algorithms interact with each other. The domain model shows how the reward function utilizes the box detection model. It also shows the DRLwithTL_real repository, which takes the weights/biases from the output network of PEDRA as its input. Note, the domain model represents how the simulation environment softwares (PEDRA, Airsim, Unreal Engine) interact with one another and how they provide a platform where we can train a simulated drone for various interactions before training a drone in the real world. This informed our decision making because understanding which softwares was necessary and how they interact with one another gives a sharper understanding of the process we have to take to train the drone and how to transfer the network that performed the training onto a real drone.

Next we will go over the individual team members' approach to their own deliverables. First we will discuss Ruben's approach to his deliverables.

Ruben's first deliverable will be to figure out how to run the DRLwithTL_real repository. His approach to this will be: study the code and try running the repository using the instructions provided in the README, tweaking things/working through errors until he gets it working. Once Sam has figured out how to transfer the weights/biases from PEDRA, we will initialize the reinforcement learning neural network inside of the DRLwithTL_real repository with these weights/biases and then run the repository. Ruben's next deliverable will be to work with Sam to get PEDRA training (solve dependencies, etc.). Ruben's approach to do this will be: continue to work through errors with Sam doing peer programming since it seemed to work very well last time. The errors that are left are all related to the Tensorflow code in the repository being out of date, so we will look up the updated syntax to update the code, as well as utilizing scripts that update the tensorflow code for us like we have been doing. Ruben's final deliverable will be to figure out how to incorporate a box detection model into reward functions, then make sure it has the desired/theorized effect on the drone. Ruben's approach for this will be: the box detection model will be utilized in order to know whether there is a box in front of the drone or not, if there is a box in front of the drone and the drone is currently going straight, then give the drone a positive reward since it's going in the correct direction. Ruben needs to figure out how to turn this into code. After we have trained the box detection model, we will incorporate it into the reward function in the manner described using Tensorflow. We will then test to see if this has the desired effect on the drone (the drone is able to fly around without crashing but favoring going near the boxes - thereby monitoring the boxes).

Next we will discuss Sam's deliverables and approaches to those deliverables. The first deliverable will be performing transfer learning, figuring out exactly how to transfer the weights/biases and get it to continue training in the real world. Sam's approach to this deliverable will be: most of the training will take place in simulation environments (PEDRA). The first few layers (convolution layers) will be trained in the simulation phase and later the weights/biases will be transferred to DRLwithTL_real repository neural network where the weights/biases are initialized in the edges of the convolution layers for continuing the training for the remaining fully connected layers. This leads to faster convergence in the real time. Sam's next deliverable will be to work with Ruben to get PEDRA training (solve dependencies, etc.) (see approach above). The last deliverable will be to help with incorporating the box detection model into the reward functions. Sam's approach to this deliverable will be: after successfully navigating through the obstacles the next phase of the drone would be to detect the boxes while avoiding the obstacle. In this phase we had planned to train a separate neural network to detect boxes. The boxes are manually taken photos from Tolu and they are augmented multiple times for larger dataset. After making our model worthy of detecting the box, our last phase would be to work together in figuring out how to incorporate the box detection model with reward function in reinforcement learning.

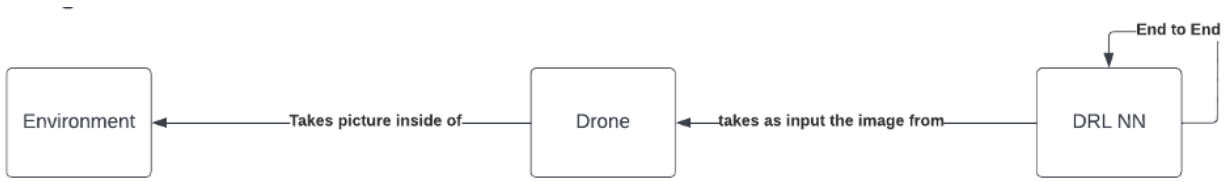
Lastly we have Tolu's deliverables. His first deliverable will be to collect data for box detection

model. His approach to this deliverable will be: manually take various pictures of boxes in different sizes and colors at different angles. Perhaps include an image augmentation algorithm and label all the boxes, or implement more images from ImageNet. Tolu's next deliverable will be to train box detection model (SSD — single shot detector). His approach for this will be: Implement a Single-Shot Detector (SSD) model. [22] This network extracts semantic meaning from input images while maintaining their images' spatial structure. [22] It implements image grids to detect objects and anchor boxes to match images and find ground truth. This algorithm also takes into account the aspect ratios, and to maintain ground truth. [22] We will utilize the transfer learning models to use the ImageNet weights, as this will allow us to get our model to converge with fewer iterations than it otherwise would. Tolu's next deliverable is to figure out what we need to do to fly the Tello drone. His approach for this is: speak with RIT's campus facilities/Imaging Science department professors regarding where this can be done. His final deliverable is to help with incorporating the box detection model into the reward functions. His approach for this is: implement a sequential detection model to locate the target with a minimal number of evaluations. This will depend on finding a time step during detection sequences where each (fixate) action depends on its (internal) state. [23] The main component is to focus on parts of the image with "richer" pieces of information and zoom in on them. [23] The agent will be trained in a way that it is given an image window and becomes capable of deciding where to focus attention among different region candidates (small windows). [23] This is the process of performing a hierarchical image analysis. [23]

5 Implementation:

5.1 Architecture:

5.1.1 Ingestion Layer:



Data Ingestion

In our project, the drone takes regular photos of its environment while it is flying in the air. The Deep Reinforcement Learning Neural Network takes these photos as input, one by one, training (end to end) on each one in an online fashion.

The only dataset we are utilizing is a dataset composed of pictures of boxes. We use this dataset to train the box detection model. The images are of the .png file type as this is what was used in the PEDRA repository that we are using. This is the only dataset we are using since the drone learns how to navigate its environment through the online process of reinforcement learning.

We are collecting the data to train the box detector model ourselves by taking pictures of boxes in the environment that we will be training the drone in.

Our data needs to be annotated, and accordingly we will mark the images that we collect where the boxes are so that the Single Shot Detector model we are implementing can learn how to detect boxes.

Our box data will be stored in a directory labeled "picturesOfBoxes" in one of our GitHub repositories. We will store this data in the repository that is responsible for training the drone

in the real world (it makes logical sense because we have pictures of boxes in the real world): DRLwithTL_real. The directory structure of the rest of the repository can be viewed here: https://github.com/aqeelanwar/DRLwithTL_real. We chose to store it in a folder instead of a database because we figured it was the simplest way to store the data and felt that a database was overkill.

5.1.2 Data Cleaning Layer:



Data Cleaning RL

Before feeding the image to Deep reinforcement learning neural network, the image needs to be processed. In two phases of training we have to process our image, during our first (simulation) phase of training the photo will be taken inside the simulation environment by drone and the photo will be saved during each state (each frame) in the Airsim directory. In our second phase after performing transfer learning we will be using a real drone to train further by utilizing the images taken from the real world. In both the phases the image is processed in a similar manner. Let's dive into the image processing technique.

In order to make decisions a drone needs the image from the environment. The drone camera records the environment continuously in video format, and Airsim makes it possible to obtain the single frame in RGB array string format. Further, the string array is converted to numpy array format using numpy fromstring method. Our next preprocessing step would be to convert the numpy array into three matrices each representing 227 * 227 grid with the help of numpy resize method. Then, the resized numpy array is passed to openCV normalization method. By normalizing the image all the pixel intensities are improved and segmented properly which results in better convergence. The preprocessed dataset is stored in the local file directory before moving to the modeling layer. Finally, the preprocessed image is passed to Deep reinforcement learning neural network for choosing the best pixel to move forward.

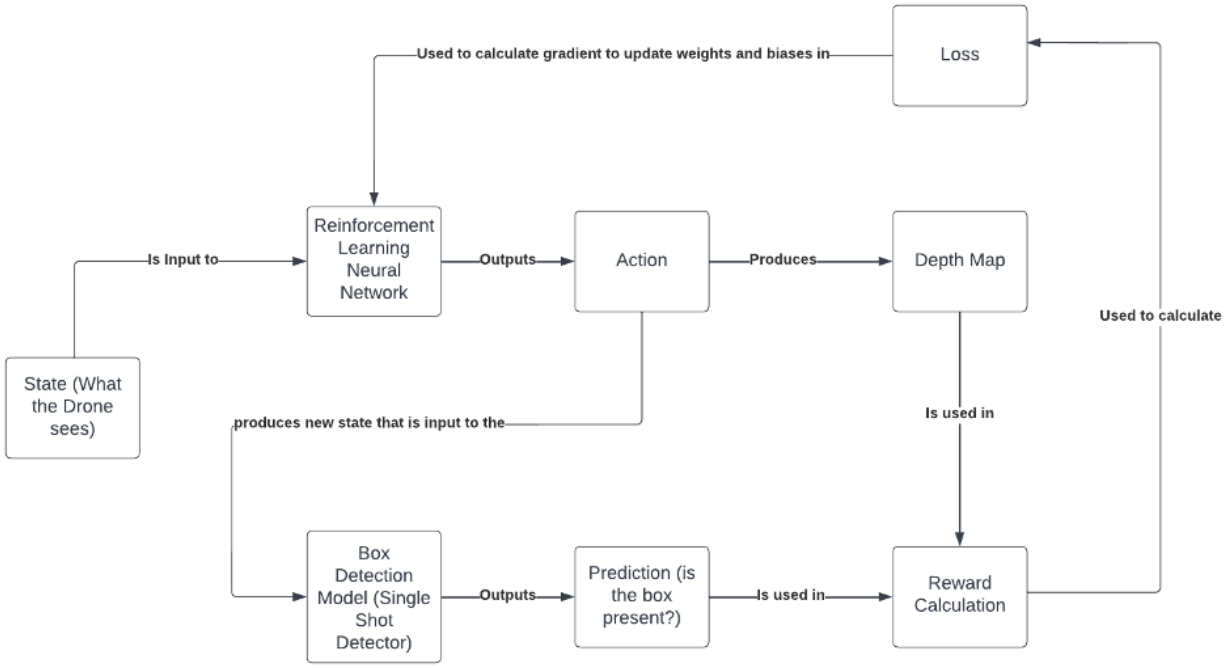


Data Cleaning box

For our next stage of image processing, we need to process the box image taken from the camera. These images are also augmented by rotating the image in different variations. Both the image forms are then resized to specific aspect ratios to reduce the memory usage or else during convolution this process will take a lot of time to train. Since we will use a Single Shot Detector model for object detection, another advantage of resizing into specific dimensions is that this will help the Single Shot Detector model to get the desired input. After resizing the dimension, the resized image is fed to the pixel brightness transformation (PBT) layer where we will apply a mathematical function to enhance the pixel contrast. This Pixel Brightness Transformation will improve a lot in prediction to identify the difference between a box and a forest. Finally, the image is fed to the normalization layer, where we standardize all the pixel values based on mean

and standard deviation. This will also help the SSD model to identify the different features in the pixel. And at last the preprocessed image is fed to the SSD model for training.

5.1.3 Modeling layer:



Modeling

The state (what the drone currently sees) is input to the reinforcement learning neural network. The state gets put through the policy network which outputs an action. After the drone takes this action, a depth map is produced, which is used in the reward calculation. For example, if the drone turns left and then it is in front of a wall, the depth map would show all dark colors (indicating that an object is right in front of it). This depth map would be used in the reward calculation function which would interpret this as a negative reward as we want to penalize collisions. Concurrently, the new state (the state after the drone takes the action) is passed to the box detection model (a Single Shot Detector model) which predicts whether there is a box in the state or not. This gets used in the reward calculation. We selected a Single Shot Detector object detection model for a few reasons: SSDs have a high speed and accuracy for low resolution images, they eliminate bounding box proposals similar to those used in Region Based Convolutional Neural Networks, and they "include progressively decreasing convolutional filter[s] for predicting object categories and offsets in bounding box locations". [1] For example, if the box is present, the drone will receive a positive reward because we want the drone to go towards the boxes. Once the reward has been calculated, it is used to calculate the loss, which is used to calculate the gradient to update the weights/biases in the reinforcement learning neural network.

For our data set, we will implement a Single Shot object detection algorithm (SSD). The SSD divides images using a grid, and then makes the grid cells responsible for detecting objects in that respective region of the image. Detecting objects then means making predictions of the location and class of the object within that particular region. [2] For the SSD algorithm we will use the algorithm on images that we collected and on images generated through data augmentation. Further, we will likely resize the images before putting them into the SSD model. For the reinforcement learning model, we will use the image data that has been resized and normalized.

What are you trying to predict? How are you quantifying success?

We aim to predict the proper classification of the images of objects in the drone's field of view for the object detection model as we want the drone to properly monitor boxes on its own. In the context of the Reinforcement Learning Algorithm, we are training the drone's policy network to take a state and predict an action that will provide the highest reward. In order to quantify successes in the object detection model, we are going to account for accuracy by calculating the amount of images that were correctly predicted of all given images.

In order to achieve success, the following must happen 5 times in a row:

1. The drone flies for 30 minutes without crashing.
2. During the flight, the drone must also stay within a 5 meter radius of any box for at least 70% of the flight.

Are you visualizing your results, if so how, and with what techniques?

We will be visualizing our results by creating a plot for our data points. For the reinforcement learning plot, the independent variable (X-Axis) will be the time steps as they relate to the agent's actions (the drone), and the dependent variable (Y-axis) will be the cumulative reward. To visualize the object detection model results, we will make plots for model accuracies against the number of images and select the best model.

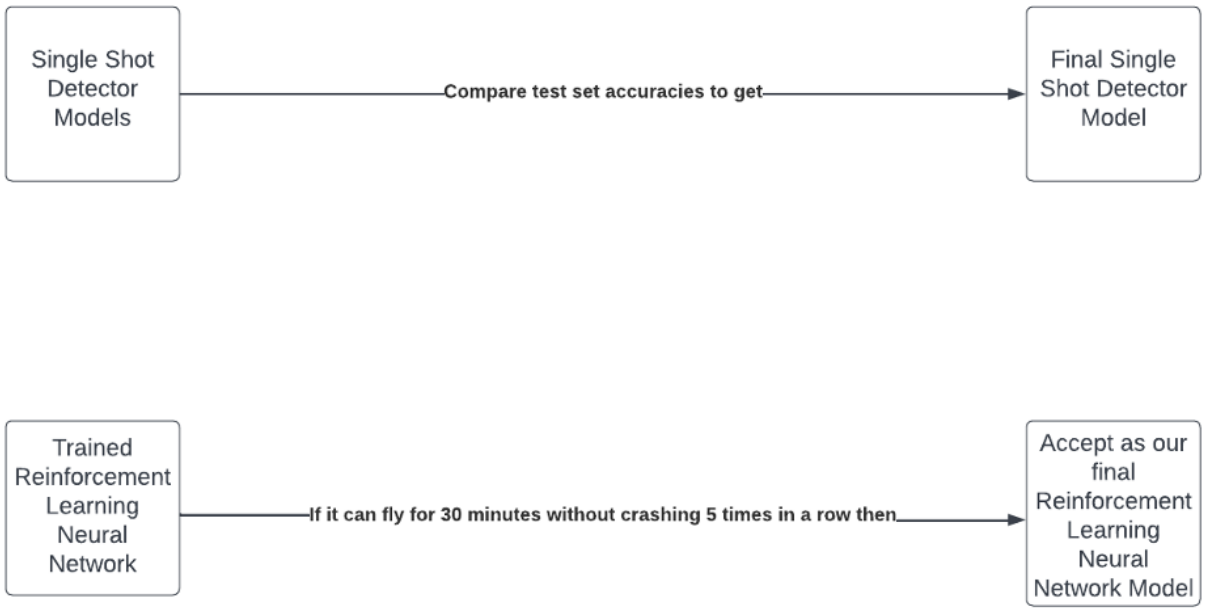
Are you re-using models from the Exploration Layer, e.g., utilizing a clustering model to detect new outliers?

We will not be using an exploration layer.

How can new data be fed into the modeling layer? Are you doing online predictions?

New data is fed into the modeling layer in an online fashion. Since we are doing reinforcement learning, the drone takes an image of its environment at a regular interval, which then gets pushed through the diagram illustrated and described above.

5.1.4 Analysis layer:



Analysis

In this project, the going will use multiple Single Shot Detection models which will have different hyper parameters including but not limited to learning rate, batch size, training epoch, and more.

To store the data from previous layers, we will create a CSV file with the name of the images, and the corresponding classification next to them. Similarly, a CSV will be created to store the values of the reward and penalty metrics for each time step that corresponds to the state of the drone.

We will get the model accuracy of each Single Shot Detection model on a test set. We implement a train test split where for each Single Shot Detection model, we get the accuracy on the test set to check the skill of the models.

After this step, we compare these test set accuracies for each one and select the model with the highest accuracy on the test set. That becomes our final SSD model.

Once we have confidence that our model can be finished, we will test the drone by flying it using the Reinforcement Learning Neural Network that has been trained so far. If the drone is able to fly for 30 minutes without collisions for 5 consecutive flights as well as during each flight, the drone must also stay within a 5 meter radius of any box for at least 70% of the flight, we accept this as our final Reinforcement Learning Neural Network model. The prediction layers will produce data in the form of file classification (files having a box or not) and predictions from the Reinforcement Learning Neural Network model will be in the form of reward/penalty metrics.

Our results will be presented to domain experts and those unfamiliar with reinforcement learning, and those without much experience with drones or reinforcement learning alike. Our results will be communicated to either party through visualizations that detail the process of rewarding and penalizing drone decisions.

Therefore, the data will be easily read by visualization tools as numerical outputs: Classification will be read as 0 and 1 outputs through one hot encoding, and reward/penalty metrics will have positive and negative values respectively. This data will be able to be visualized using

libraries such as matplotlib or seaborn.

How all the layers Interact?

Ingestion Layer → Data Cleaning Layer:

The ingestion layer and data cleaning layers interact with each other like so: the ingestion layer has to do with the dataset we are using and how we collected it, which in our case is the pictures of boxes dataset that we use to train the Single Shot Detector object detector to detect boxes in the outdoor forest environment. The data cleaning layer alters this data into a form that can be used to train/evaluate a model. So in our case our data cleaning layer takes these images and prepares them to be used to train the Single Shot Detector model. It also cleans the images that the drone takes while doing reinforcement learning so that the drone can train on them.

Data Cleaning Layer → Modeling Layer:

Once the data has been cleaned (resized and normalized as necessary) then it can be used to train a model. Upon doing so, we will be able to train each of our models using this cleaned data. Lastly, we run the models on the testing data so we can make actual predictions. In our case, we first clean the pictures of boxes dataset that we create (using the process described above), then train the box detector model on this cleaned dataset, then we run this trained box detection model on the test set and make predictions.

Modeling Layer → Analysis Layer:

The modeling layer directly interacts with the analysis later because the modeling layer establishes the Reinforcement Learning Neural Networks policy network, depth map, and reward calculations. These are among the things we will assess in our Analysis Layer, and empirically reconcile with our findings to extract a firm understanding of the performance of our algorithms as they relate to the drones performance.

In terms of the SSD algorithm, the Modeling layer uses the action to produce a new state that will be an input for the SSD. The outputs will be used to make a prediction as to whether or not a box is present, and this information will inform the reward calculation. This is important for the Analysis layer because in the analysis layer we will be comparing test accuracies to get the final SSD model that we will use.

Why we selected the different software frameworks and components?

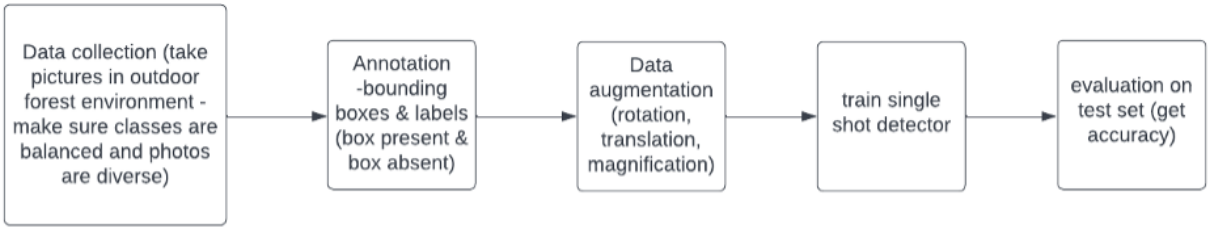
We selected PEDRA because it is an environment in which we can train a reinforcement learning neural network to train the drone we are using. It provides us with realistic 3D environment options where we can train a simulation drone. Once the Reinforcement Learning Neural Network is trained, we take these weights and biases from the Neural Network to initialize the Reinforcement Learning Neural Network inside of the DRLwithTL_real repository. In terms of how the software layers interact, Unreal Engine is the software gaming engine platform upon which PEDRA interfaces with to create the virtual-realistic flying environments. PEDRA's 3D realistic environments can be selected by us, and the PEDRA software interfaces with Microsoft's Airsim software which is an open source plugin that "interfaces Unreal Engine with Python". [3] This allows us to work with python functionalities that "control the sensory inputs and control signals of the drone". [3] Airsim allows us to work with python modules so that we can work on Reinforcement Learning methodologies as they pertain to our project. We selected Unreal Engine and Airsim as we first discovered the PEDRA software, and it utilizes Unreal Engine and Airsim software.

The last software framework we chose was the DRLwithTL_real repository. We take the output trained reinforcement learning neural network weights/biases from PEDRA and use these weights/biases to initialize the reinforcement learning neural network in DRLwithTL_real (thereby

performing transfer learning). We then continue to train using reinforcement learning in the DRLwithTL_real repository in order to train the drone to fly in the real world. This repository was made by the same person who made PEDRA so we were already familiar with the code to some degree due to this fact, which is one of the reasons we chose this software framework. Another reason is that we searched for other repositories that did this same thing and couldn't find any. The final reason is that the repository uses the same neural network architecture as PEDRA, which is essential for transfer learning to work correctly.

We chose the Single Shot Detector model because we read in this [4] paper that it was the best object detection model. The paper tested many object detection models against each other.

Technical Diagram of Single Shot Detector (SSD) training and evaluation process:



Technical Diagram

The training process for the single shot detector box detection model is illustrated above. The process proceeds as follows: first we collect many pictures in an outdoor forest environment, some of these photos will have boxes in them and others won't. We make sure that the two classes (box present and box absent) are balanced in the data we collect and that the photos are diverse (pictures of boxes are taken at different angles, maybe some of them are partially blocked by a tree, etc.). We then perform annotations. We draw bounding boxes and label them as either box present or box absent. Next we perform data augmentation in order to get more variety in our data as well as to increase the amount of data we are training on. We do this by performing rotations, translations, and magnifications of the original images. We then train the Single Shot Detector model on this data and evaluate its performance by getting its accuracy on the test set (which we would have previously collected using the same method in the data collection step).

6 Data sets

The first "dataset" is the data that the drone collects while navigating and learning from its environment using reinforcement learning. The drone regularly takes photos using its front camera, these pictures are utilized in reinforcement learning. These photos are in a forest environment since that is where our drone learns to navigate. The drone first does this in a virtual environment in Unreal Engine, and then does the same thing in the real world. The second and final dataset that we utilize is a manually annotated dataset of boxes in a real (not simulated) forest environment. This dataset is necessary in order to train our box detection model which is a Single Shot Detector model. We take photos where a box is present in the picture and we also take shots where the box is not present, as the Single Shot Detector needs to tell us whether or not a box is present in the picture that the drone sees. Therefore, the two annotation labels are "box present" and "box not present". We manually label this dataset by drawing bounding boxes around the box in each picture and labeling it "box present" so that the model is able to detect boxes. And if there is no box present then we label it as not having a box. We do this using the tutorial

here [28]. Using data augmentation techniques such as magnification, translation, and rotation, we create a dataset that has 500 pictures with boxes in them and 500 pictures without boxes in them. We take around 250 pictures for each of these 2 categories and then use data augmentation techniques to generate the rest of the pictures. We train our model on 80% of this data and test with the remaining 20%. We use accuracy as our metric in order to determine the performance of the model and of course make sure that our classes are balanced so that accuracy may be used. We shoot for at least 85% accuracy.

7 Future Work

We will complete working with Airsim software and successfully work with simulating drone movements in the environments available. Our group has yet to complete the reinforcement learning network training or the transfer learning components of the project, and those are things we will develop in the first few weeks of the upcoming semester. We will also work diligently over break to make good progress. More specifically, we will use transfer learning to transfer the weights from the reinforcement learning network that were trained on the simulation environment onto the real time drone. Further, we will set up the real time environment wherein we will continue training the drone after the transfer learning process, so that we can develop and update our object monitoring and surveillance algorithm. We will test the drone's accuracy and efficacy.

8 Conclusion

Overall, the main goal of our project was to implement a reinforcement learning algorithm to autonomously avoid obstacles and search and monitor an object (in this case it's a box). To achieve the above-mentioned goal the following works have been done this semester 1) Invested one month of our time to go through the papers and current trends in the project. Our plans of work were changed from the start due to complications in implementing the project. 2) Once we had identified the project (mentioned above) we faced resource constraints to install the softwares (in this scenario the project requires a minimum requirement of high-end GPU to train the drone in a simulation environment and our group faced a tough time in interfacing Airsim and Unreal Engine in our hardware). 3) After successfully tackling the hardware problems, we tested a sample code to check Airsim and Unreal Engine are properly interfaced. Then we found a Github repository (PEDRA) that was already implemented to train a drone using reinforcement learning in a simulated environment. The repository was not maintained and had a lot of issues. We resolved the library dependency issues but later found out the TensorFlow code was written using version 1, and the entire neural network module needed to be updated. We read the code line by line and updated the newer version of TensorFlow, but this took a lot of our time because the code still had dependency issues and we had to come up with a new idea again 4) After multiple failures of not progressing in our project due to issues in the repository code we were working with, we decided to implement our own reinforcement learning code to run in the simulated environment. We invested two more weeks in gaining the Reinforcement learning foundational basics and implemented different RL projects like Atari, racing, and cart pole games to understand the policy gradient, RL algorithm, reward function, and neural network purpose in the RL. 5) At last, we wrote preliminary reinforcement learning code in python to make our drone fly in certain altitude range where the reward will be maximized. The preliminary results are really important to make sure our drone is learning using reinforcement learning because our work will be build upon top of this. We might revisit the use of PEDRA in the future but are not sure if we will end up using it.

9 Works Cited

[1] Anwar, Aqeel Raychowdhury, Arijit. (2018). NAVREN-RL: Learning to fly in real environment via end-to-end deep reinforcement learning using monocular images.

[2] I. Yoon, M. A. Anwar, R. V. Joshi, T. Rakshit and A. Raychowdhury, "Hierarchical Memory System With STT-MRAM and SRAM to Support Transfer and Real-Time Reinforcement Learning in Autonomous Drones," in IEEE Journal on Emerging and Selected Topics in Circuits and Systems, vol. 9, no. 3, pp. 485-497, Sept. 2019, doi: 10.1109/JETCAS.2019.2932285.

[3] Venturini, Federico Mason, Federico Pase, Francesco Chiariotti, Federico Testolin, Alberto Zanella, Andrea Zorzi, Michele. (2020). Distributed reinforcement learning for flexible UAV swarm control with transfer learning capabilities. 1-6. 10.1145/3396864.3399701.

[4] A. Anwar and A. Raychowdhury, "Autonomous Navigation via Deep Reinforcement Learning for Resource Constraint Edge Nodes Using Transfer Learning," in IEEE Access, vol. 8, pp. 26549-26560, 2020, doi: 10.1109/ACCESS.2020.2971172.

[5] Unreal Engine. Accessed: Dec. 2, 2019. [Online]. Available: https://www.unrealengine.com/en-us/

[6] H. X. Pham, H. M. La, D. Feil-Seifer, and L. V. Nguyen, "Autonomous UAV navigation using reinforcement learning," CoRR, vol. abs/1801.05086, 2018.

[7] Cetin, E. et al. Drone Navigation and Avoidance of Obstacles Through Deep Reinforcement Learning. Escola dEnginyeria en Telecomunicacions i Aerospacial de Castelldefels Universitat Politecnica de Catalunya, BarcelonaTECH. Available at:
https://upcommons.upc.edu/bitstream/handle/2117/186572/postprint_DOI_10_1109_DASC43569_2019_9081749.pdf?sequence=3 (Accessed: October 19, 2022)

[8] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. Van Den Driessche, J. Schrittwieser, I. Antonoglou, V. Panneershelvam, M. Lanctot, S.Dieleman, D. Grewe, J. Nham, N. Kalchbrenner, I.Sutskever, T. Lillicrap, M. Leach, K. Kavukcuoglu, T. Graepel, and D. Hassabis, "Mastering the game of go with deep neural networks and tree search," Nature, vol. 529, no. 7587, pp. 484–489, Jan. 2016.

[9] V. Mnih, K. Kavukcuoglu, D. Silver, A. A. Rusu, J. Veness, M. G. Bellemare, A. Graves, M. Riedmiller, et al., "Human-level control through deep reinforcement learning," Nature, vol. 518, no. 7540, pp. 529–533, 2015.

[10] D. Silver, A. Huang, C. J. Maddison, A. Guez, L. Sifre, G. van den Driessche, J. Schrittwieser, I. Antonoglou, et al., "Mastering the game of go with deep neural net- works and tree search," Nature, vol. 529, no. 7587, pp. 484–489, 2016.

[11] M. Campbell, A. J. Hoane, and F. Hsu, "Deep Blue," Artificial Intell., vol. 134, no. 1-2, pp. 57–83, 2002

[12] R. S. Sutton and A. G. Barto, Reinforcement Learning: An Introduction. Cambridge, MA: MIT Press, 1998.

[13] A. Rummery and M. Niranjan, "On-line Q-learning using connectionist sys- tems," Dept. Engineering, Univ. Cambridge, MA, Tech. Rep. CUED/F-INFENG/ TR 166, 1994.

[14] V. R. Konda and J. N. Tsitsiklis, "On actor-critic algorithms," SIAM J. Control Optim., vol. 42, no. 4, pp. 1143–1166, 2003.

[15] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High- dimensional continuous control using generalized advantage estimation," in Proc. Int. Conf. Learning Representations, 2016.

[16] Y. Song, M. Steinweg, E. Kaufmann, and D. Scaramuzza, Autonomous Drone Racing with Deep Reinforcement Learning, 2021. [Online]. Available: https://rpg.ifi.uzh.ch/docs/IROS21_Yunlong.pdf. [Accessed: 18-Oct-2022].

[17] Z. Xue and T. Gonsalves, "Vision Based Drone Obstacle Avoidance by Deep Reinforcement Learning," MDPI, 07-May-2021. [Online]. Available: https://www.mdpi.com/2673-

2688/2/3/23. [Accessed: 19-Oct-2022].

[18] Mnih, V., Kavukcuoglu, K., Silver, D., Rusu, A.A., Veness, J., Bellemare, M.G., Graves, A., Riedmiller, M.A., Fidjeland, A., Ostrovski, G., Petersen, S., Beattie, C., Sadik, A., Antonoglou, I., King, H., Kumaran, D., Wierstra, D., Legg, S., Hassabis, D. (2015). Human-level control through deep reinforcement learning. Nature, 518, 529-533.

[19] V. N. Nguyen, R. Jenssen and D. Roverso, "Intelligent Monitoring and Inspection of Power Line Components Powered by UAVs and Deep Learning," in IEEE Power and Energy Technology Systems Journal, vol. 6, no. 1, pp. 11-21, March 2019, doi: 10.1109/JPETS.2018.2881429.

[20] Wikimedia Foundation. (2022, October 30). Airsim. Wikipedia. Retrieved November 13, 2022, from https://en.wikipedia.org/wiki/AirSim

[21] Wikimedia Foundation. (2022, November 7). Unreal Engine. Wikipedia. Retrieved November 13, 2022, from https://en.wikipedia.org/wiki/Unreal Engine

[22] ArcGIS. (n.d.). How single-shot detector (SSD) works? ArcGIS API for Python. Retrieved November 13, 2022, from https://developers.arcgis.com/python/guide/how-ssd-works/

[23] Mathe, S., Pirinen, A., amp; Sminchisescu, C. (1970, January 1). Reinforcement learning for visual object detection. CVF Open Access. Retrieved November 13, 2022, from

https://openaccess.thecvf.com/content cvpr 2016/html/Mathe Reinforcement Learning for CVPR 2016 paper.html

[24] Khandelwal, R. (n.d.). SSD : Single shot detector for object detection using MultiBox. Towards Data Science. Retrieved November 17, 2022, from https://towardsdatascience.com/ssd-single-shot-detector-for-object-detection-using-multibox-1818603644ca

[25] ArcGIS. (n.d.). How single-shot detector (SSD) works? ArcGIS API for Python. Retrieved November 13, 2022, from https://developers.arcgis.com/python/guide/how-ssd-works/

[26] Anwar, A. (n.d.). Programmable engine for drone reinforcement learning (RL) applications (pedra-2.0). PEDRA. Retrieved November 16, 2022, from https://aqeelanwar.github.io/PEDRA/

[27] Nguyen, V. N., Jenssen, R., Roverso, D. (n.d.). Intelligent Monitoring and Inspection of Power Line Components Powered by UAVs and Deep Learning. IEEE Power and Energy Technology Systems Journal. Retrieved November 16, 2022, from https://ieeexplore.ieee.org/stamp/stamp.jsp?tp=arnumber=8610301tag=1

[28] Renotte, N. (2020, November 1). Real time face mask detection with tensorflow and Python — Custom Object Detection W/ mobilenet SSD. YouTube. Retrieved December 3, 2022, from https://www.youtube.com/watch?v=IOI0o3Cxv9Qamp;t=752s